

**LAPORAN PRAKTIKUM  
APLIKASI BERBASIS PLATFORM**

**MODUL - 1, 2, 3  
PENGENALAN FLUTTER dan DART**



**Disusun oleh:**

**Wahyu Bagus Setiawan  
2311102296  
S1-IF-11-04**

**Dosen Pengampu:  
Cahyo Prihantoro, S.Kom., M.Eng.**

**PROGRAM STUDI S1 INFORMATIKA DIREKTORAT KAMPUS  
PURWOKERTO – UNIVERSITAS TELKOM MARET 2026**

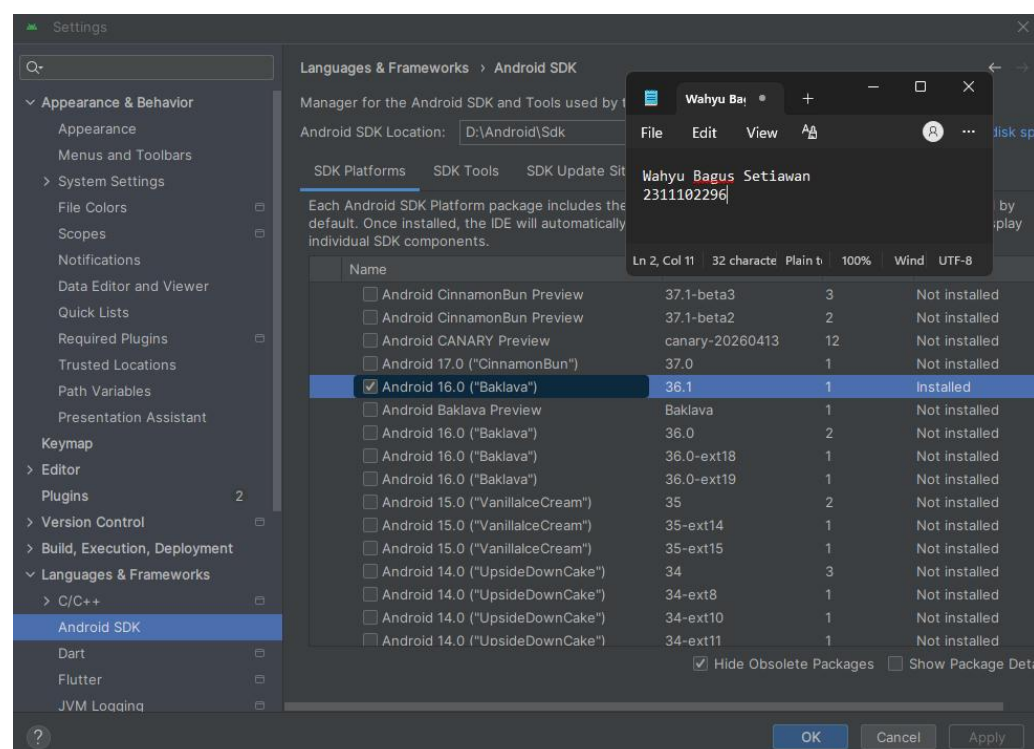
## Dasar Teori

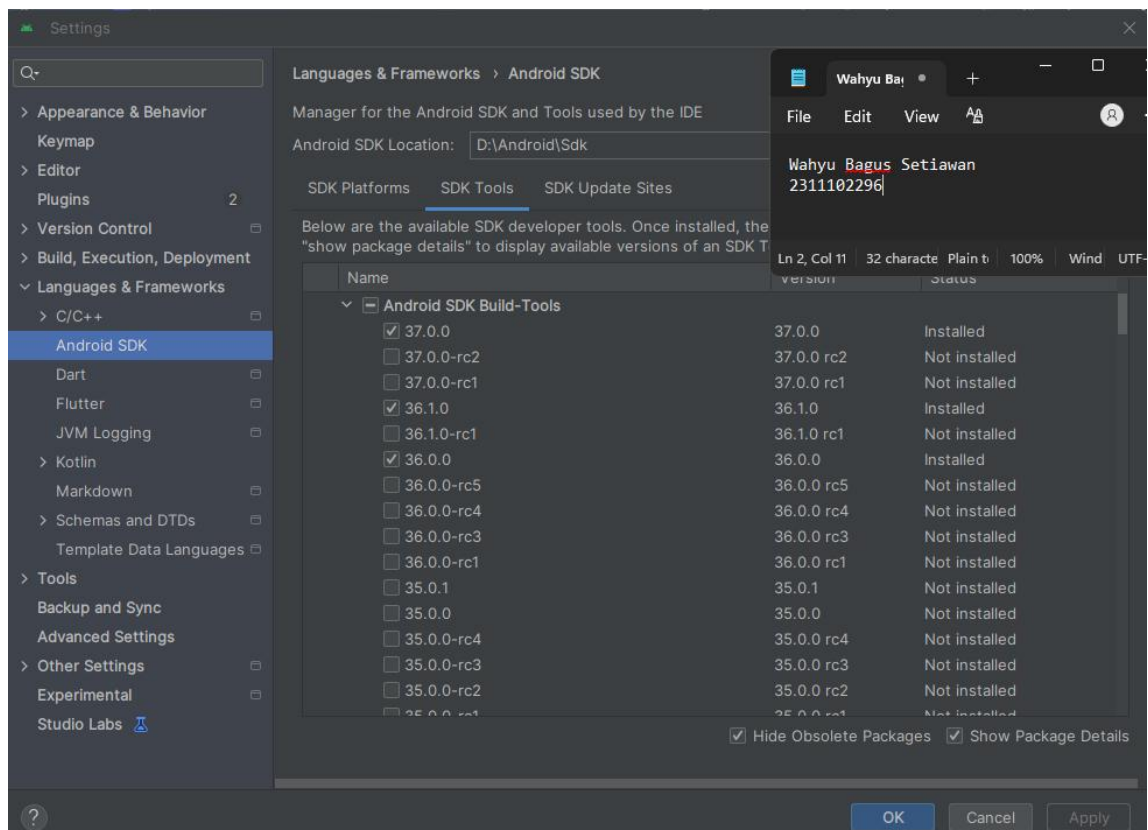
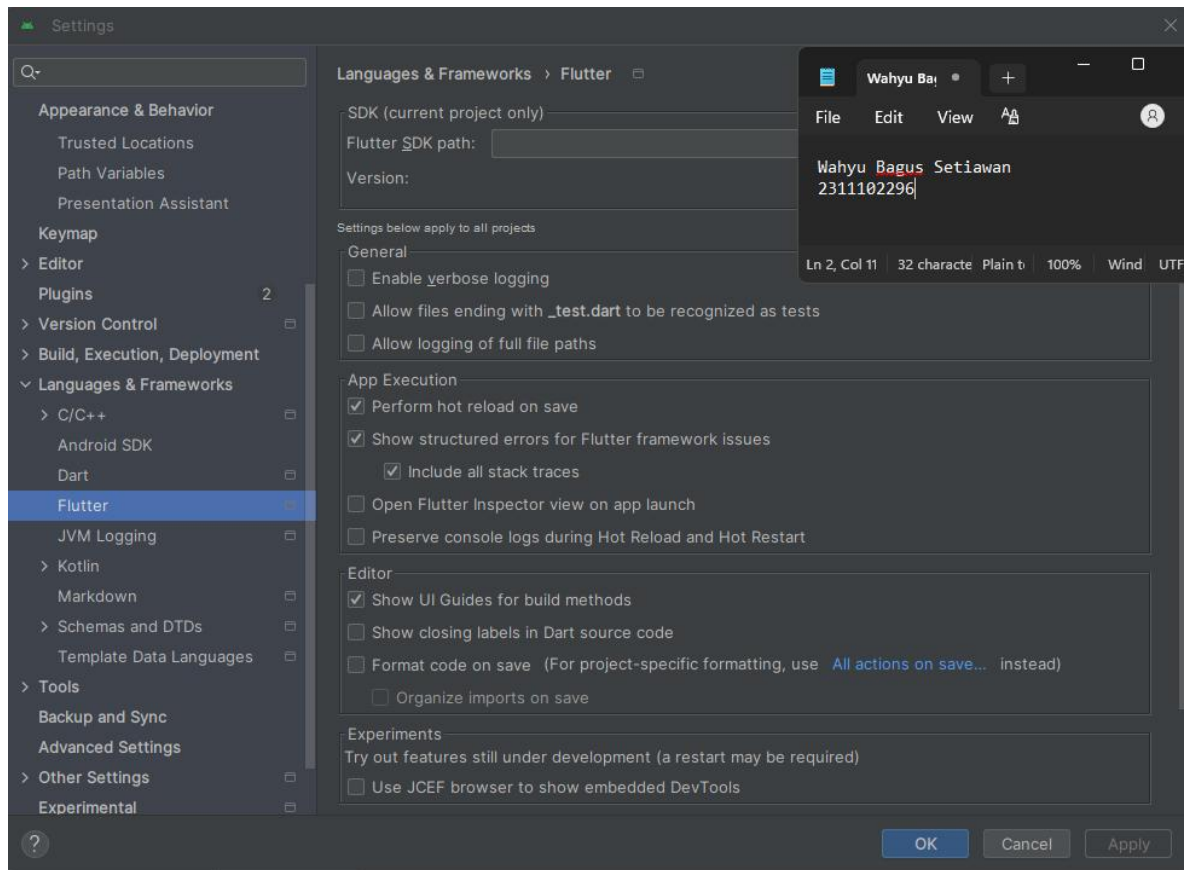
Flutter merupakan sebuah framework open-source besutan Google yang dirancang untuk memfasilitasi pembuatan aplikasi multi-platform (mobile, web, dan desktop) berbasis satu codebase. Dengan memanfaatkan bahasa pemrograman Dart dan Skia Graphics Engine, Flutter mampu merender antarmuka secara mandiri tanpa ketergantungan pada komponen bawaan platform (native). Keunggulan menonjol dari framework ini adalah adanya fitur hot reload, yang mempercepat siklus pengembangan karena developer dapat langsung meninjau perubahan kode tanpa proses build ulang.

Dalam menyusun tampilan, Flutter menerapkan konsep hirarki yang disebut widget tree. Seluruh komponen UI dikategorikan ke dalam dua jenis widget utama: stateless widget untuk elemen dengan data statis, dan stateful widget untuk komponen yang datanya dapat berubah secara dinamis saat aplikasi dieksekusi. Fondasi awal aplikasi Flutter umumnya diawali dengan MaterialApp sebagai akar (root), diikuti oleh Scaffold sebagai struktur layout utama yang menyediakan AppBar dan body, serta widget pendukung seperti Text dan Center untuk memosisikan konten.

Dari aspek manajemen arsitektur, pola BLoC (Business Logic Component) sering diimplementasikan untuk memisahkan logika bisnis dari lapisan presentasi (UI) menggunakan mekanisme event dan state. Pendekatan ini membuat kode program menjadi lebih rapi, adaptif (scalable), serta mudah diuji (testable). Sebagai tahap pengenalan awal, pembuatan aplikasi sederhana semacam "Hello World" biasanya digunakan untuk memberikan pemahaman mendasar mengenai arsitektur Flutter dan mekanisme kerja widget.

## Screenshoot Tampilan Environment & Hasil





## Screenshoot Struktur Proyek Baru

```
PS D:\Kuliah Semester 6\Aplikasi Berbasis Platform\Laprak_Pertemuan_8> flutter create hello_world
```

A new version of Flutter is available!  
To update to the latest version, run "flutter upgrade".

Creating project hello\_world...  
Resolving dependencies in `hello\_world`... (1.1s)  
Downloading packages...  
Got dependencies in `hello\_world`.  
Wrote 131 files.  
  
All done!  
You can find general documentation for Flutter at: <https://docs.flutter.dev/>  
Detailed API documentation is available at: <https://api.flutter.dev/>  
If you prefer video documentation, consider: <https://www.youtube.com/c/flutterdev>  
  
In order to run your application, type:  
  
\$ cd hello\_world  
\$ flutter run  
  
Your application code is in hello\_world\lib\main.dart.  
  
PS D:\Kuliah Semester 6\Aplikasi Berbasis Platform\Laprak\_Pertemuan\_8>

## Verifikasi Instalasi Flutter (Flutter Doctor)

(Penjelasan: Screenshot terminal hasil *flutter doctor -v* untuk memastikan seluruh dependensi terinstal dengan benar dan aman dari celah environment)

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ACER>flutter doctor -v
```

A new version of Flutter is available!  
To update to the latest version, run "flutter upgrade".

[✓] Flutter (Channel stable, 3.41.6, on Microsoft Windows [Version 10.0.26200.8457], locale en-ID) [1,059ms]  
• Flutter version 3.41.6 on channel stable at D:\Flutter\Flutter\_windows\_3.3.9-stable\Flutter  
• Upstream repository https://github.com/flutter/flutter.git  
• Framework revision db50e20168 (9 weeks ago), 2026-03-25 16:21:00 -0700  
• Engine revision 425cfb54d0  
• Dart version 3.11.4  
• DevTools version 2.54.2  
• Feature flags: enable-web, enable-linux-desktop, enable-macos-desktop, enable-windows-desktop, enable-android, enable-ios, cli-animations, enable-native-assets, omit-legacy-version-file, enable-lldb-debugging, enable-uiscene-migration  
  
[✓] Windows Version (11 Home Single Language 64-bit, 25H2, 20H9) [2.7s]  
  
[✓] Android toolchain - develop for Android devices (Android SDK version 35.0.0) [21.5s]  
• Android SDK at C:\Users\ACER\AppData\Local\Android\jdk  
• Emulator version 30.9.5.0 (build\_id 7820599) (CL:N/A)  
• Platform android-36, build-tools 35.0.0  
• Java binary at: D:\Android\Android Studio\jbr\bin\java  
This is the JDK bundled with the latest Android Studio installation on this machine.  
To manually set the JDK path, use: 'flutter config --jdk-dir="path/to/jdk"'.  
• Java version OpenJDK Runtime Environment (build 21.0.10+14961533-b1163.108)  
• All Android licenses accepted.  
  
[✓] Chrome - develop for the web [455ms]  
• Chrome at C:\Program Files (x86)\Google\Chrome\Application\chrome.exe  
  
[!] Visual Studio - develop Windows apps (Visual Studio Community 2022 17.0.4) [452ms]

```
[!] Visual Studio - develop Windows apps (Visual Studio Community 2022 17.0.4) [452ms]
• Visual Studio at C:\Program Files\Microsoft Visual Studio\2022\Community
• Visual Studio Community 2022 version 17.0.32014.148
X The current Visual Studio installation is incomplete.
  Please use Visual Studio Installer to complete the installation or reinstall Visual Studio.

[✓] Connected device (3 available) [2.6s]
• Windows (desktop) • windows • windows-x64 • Microsoft Windows [Version 10.0.26200.8457]
• Chrome (web) • chrome • web-javascript • Google Chrome 148.0.7778.179
• Edge (web) • edge • web-javascript • Microsoft Edge 148.0.3967.83

[✓] Network resources [1,713ms]
• All expected network resources are available.

! Doctor found issues in 1 category.

C:\Users\ACER>
```

## Source Code Hello World

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        body: Center(
          child: Text(
            'Hello World',
            style: TextStyle(fontSize: 28, fontWeight: FontWeight.bold),
          ),
        ),
      ),
    );
  }
}
```

## Output:

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

D:\Kuliah Semester 6\Aplikasi Berbasis Platform\Laprak_Pertemuan_8\hello_world>Flutter run
Connected devices:
Windows (desktop) • windows • windows-x64 • Microsoft Windows [Version 10.0.26200.8457]
Chrome (web) • chrome • web-javascript • Google Chrome 148.0.7778.179
Edge (web) • edge • web-javascript • Microsoft Edge 148.0.3967.83
[1]: Windows (windows)
[2]: Chrome (chrome)
[3]: Edge (edge)
Please choose one (or "q" to quit): 2
Launching lib/main.dart on Chrome in debug mode...
Waiting for connection from debug service on Chrome... 34.8s

Flutter run key commands.
r Hot reload.
R Hot restart.
h List all available interactive commands.
d Detach (terminate "flutter run" but leave application running).
c Clear the screen
q Quit (terminate the application on the device).

Debug service listening on ws://127.0.0.1:50207/ZUJAC6s4RSQ=/ws
A Dart VM Service on Chrome is available at: http://127.0.0.1:50207/ZUJAC6s4RSQ=/ws
The Flutter DevTools debugger and profiler on Chrome is available at:
http://127.0.0.1:50207/ZUJAC6s4RSQ=/devtools/?uri=ws://127.0.0.1:50207/ZUJAC6s4RSQ=/ws
Starting application from main method in: org-dartlang-app:/web_entrypoint.dart
```

## MODUL 3 - PENGENALAN DART

### - Pengenalan Dart

Untuk belajar Flutter, tidak perlu terlalu fasih mempelajari bahasa Dart secara mendalam di awal. Terdapat fundamental yang perlu dipelajari seperti variable, statement control, looping, array, fungsi, dan sebagainya. Karakteristik bahasa Dart mirip dengan bahasa C atau Java, di mana penggunaan titik koma (;) diakhir baris kodingan adalah wajib.

### -Variable

Penggunaan variable di Dart dapat dilakukan dengan beberapa cara, yaitu menggunakan var, type annotation, dan multiple variable.

Contoh Kode:

```
void main() {  
  print('=====');  
  print('      MODUL 3 - PENGENALAN DART      ');  
  print('=====');  
  
  // 1. Contoh Variable menggunakan 'var'  
  var namaVariable = "Nilai Contoh";  
  print('Contoh var      : $namaVariable');  
  // 2. Contoh Type Annotation (Perhatikan huruf kapital 'int' harus  
kecil, 'String' kapital)  
  String nama = "WahyuBagus";  
  int umur = 20;  
  print('Type Annotation   : Nama = $nama, Umur = $umur');  
  // 3. Contoh Multiple Variable  
  var a = 1, b = 2, c = 3;  
  print('Multiple Variable  : a = $a, b = $b, c = $c');  
  
  print('=====');  
}
```

Variable primitif yang tersedia di Dart: Integer (int), Double (double), String (string), Boolean (bool)

Screenshoot Output:

```
=====  
      MODUL 3 - PENGENALAN DART        
=====  
Contoh var      : Nilai Contoh  
Type Annotation   : Nama = WahyuBagus, Umur = 20  
Multiple Variable  : a = 1, b = 2, c = 3  
=====
```

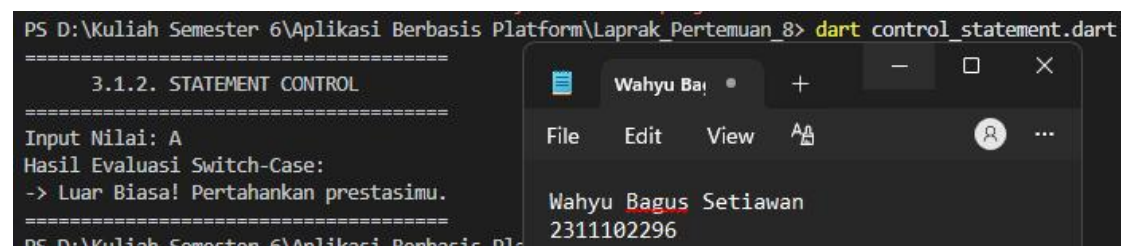


### - Statement Control

Statement control digunakan untuk menentukan alur eksekusi program. Dart mendukung if, if-else, if-else-if, dan switch-case. Contoh Struktur Switch Case:

```
void main() {  
  print('=====');  
  print('      3.1.2. STATEMENT CONTROL      ');  
  print('=====');  
  
  String nilaiHuruf = 'A';  
  print('Input Nilai: $nilaiHuruf');  
  print('Hasil Evaluasi Switch-Case:');  
  // Struktur Switch Case sesuai Modul 3  
  switch (nilaiHuruf) {  
    case 'A':  
      print('-> Luar Biasa! Pertahankan prestasimu.');      break;  
    case 'B':  
      print('-> Bagus! Tingkatkan lagi belajarnya.');      break;  
    case 'C':  
      print('-> Cukup. Jangan lupa dievaluasi ya.');      break;  
    default:  
      print('-> Nilai tidak valid atau perlu mengulang.');      break;  
  }  
  print('=====');}
```

Screenshot Output:



```
PS D:\Kuliah Semester 6\Aplikasi Berbasis Platform\Laprak_Pertemuan_8> dart control_statement.dart  
=====  
      3.1.2. STATEMENT CONTROL        
=====  
Input Nilai: A  
Hasil Evaluasi Switch-Case:  
-> Luar Biasa! Pertahankan prestasimu.  
=====
```

## - Looping

Dart menyediakan dua cara mendasar untuk melakukan struktur perulangan, yaitu:

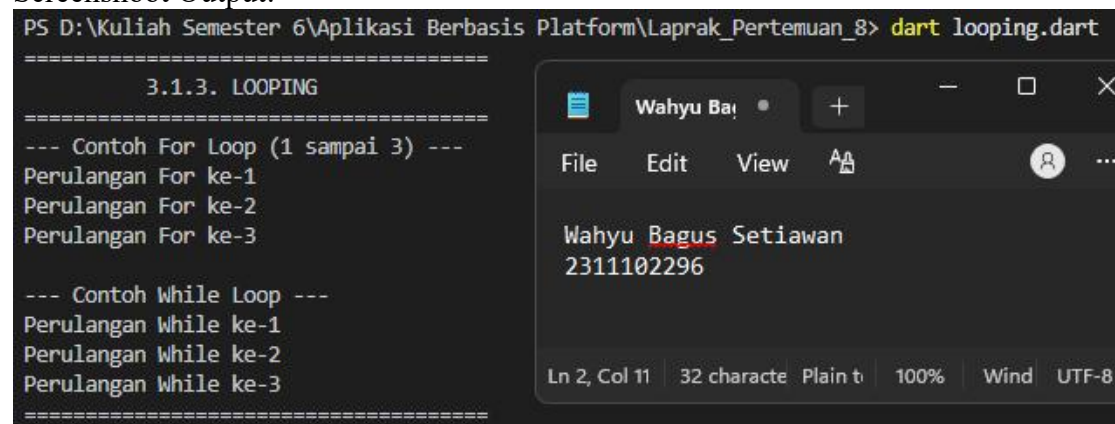
Struktur For: Sangat ideal digunakan jika kita sudah tahu persis berapa kali sebuah instruksi harus diulang (berdasarkan nilai pencacah/konter).

Struktur While: Digunakan untuk situasi di mana perulangan harus terus berjalan selama ekspresi atau kondisi logisnya bernilai benar (true), tanpa tahu batas pastinya.

```
void main() {
  print('=====');
  print('          3.1.3. LOOPING          ');
  print('=====');

  // 1. FOR LOOP (Jumlah perulangan sudah diketahui pasti, misal: 1
sampai 3)
  print('--- Contoh For Loop (1 sampai 3) ---');
  for (int i = 1; i <= 3; i++) {
    print('Perulangan For ke-$i');
  }
  print(''); // Baris baru pembatas
  // 2. WHILE LOOP (Berdasarkan kondisi logika, misal: jalan selama
angka kurang dari 3)
  print('--- Contoh While Loop ---');
  int angka = 1;
  while (angka <= 3) {
    print('Perulangan While ke-$angka');
    angka++; // Increment agar tidak terjadi infinite loop
  }
  print('=====');
}
```

## Screenshoot Output:



```
PS D:\Kuliah Semester 6\Aplikasi Berbasis Platform\Laprak_Pertemuan_8> dart looping.dart
=====
          3.1.3. LOOPING
=====
--- Contoh For Loop (1 sampai 3) ---
Perulangan For ke-1
Perulangan For ke-2
Perulangan For ke-3
--- Contoh While Loop ---
Perulangan While ke-1
Perulangan While ke-2
Perulangan While ke-3
=====
```



## - List

Struktur Data List Kumpulan objek atau data sejenis yang disimpan dalam satu variabel terstruktur di Dart dikenal dengan istilah List (analog dengan Array pada bahasa pemrograman lain). Terdapat dua jenis List utama yang digunakan:

**Fixed Length List:** Jenis List yang ukuran atau kapasitas indeksnya sudah dikunci sejak awal pembuatan, sehingga jumlah elemen di dalamnya tidak bisa ditambah atau dikurangi.

**Growable List:** Jenis List dinamis yang ukurannya fleksibel, di mana kita bisa menambah atau menghapus elemen secara bebas sesuai kebutuhan program saat berjalan.

```
void main() {
  print('=====');
  print('          3.1.4. LIST          ');
  print('=====');

  // 1. FIXED LENGTH LIST (Panjangnya dikunci: 3 elemen)
  print('--- Contoh Fixed Length List ---');
  var fixedList = List.filled(3, 0); // Membuat list berisi 3 angka nol
  fixedList[0] = 12;                 // Mengisi indeks ke-0 dengan
angka 12
  fixedList[1] = 25;
  fixedList[2] = 50;
  print('Isi Fixed List : $fixedList');
  print('Panjang List   : ${fixedList.length}');
  print(''); // Pembatas
  // 2. GROWABLE LIST (Bisa bertambah terus ukurannya)
  print('--- Contoh Growable List ---');
  var growableList = []; // Membuat list kosong dinamis
  growableList.add(12);  // Menambah elemen pertama
  growableList.add(24);  // Menambah elemen kedua
  growableList.add(36);  // Menambah elemen ketiga
  print('Isi Growable List: $growableList');
  print('Panjang List     : ${growableList.length}');
  print('=====');
}
```

## Screenshoot Output:

The screenshot shows the output of the Dart program in a terminal window. The output is as follows:

```
PS D:\Kuliah Semester 6\Aplikasi Berbasis Platform\Laprak_Pertemuan_8> dart list_praktikum.dart
=====
          3.1.4. LIST
=====
--- Contoh Fixed Length List ---
Isi Fixed List : [12, 25, 50]
Panjang List   : 3

--- Contoh Growable List ---
Isi Growable List: [12, 24, 36]
Panjang List     : 3
=====
```

Overlaid on the right side of the terminal is a window titled 'Wahyu Bai' containing the text:

Wahyu Bagus Setiawan  
2311102296

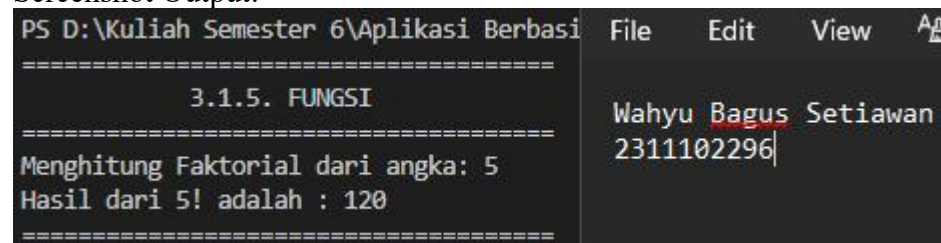
### - Fungsi

Di dalam paradigma pemrograman, fungsi (function) memegang peranan krusial untuk mengimplementasikan konsep Separation of Concerns (pemisahan tanggung jawab kode). Melalui pemanfaatan fungsi yang spesifik, struktur program menjadi lebih modular, meminimalkan adanya duplikasi kode (boilerplate code), serta lebih mudah untuk dirawat. Salah satu contoh penerapannya adalah Fungsi Rekursif, yaitu sebuah fungsi yang memanggil dirinya sendiri secara berulang untuk menyelesaikan suatu kasus (seperti perhitungan faktorial) hingga mencapai kondisi batas (base case) tertentu.

```
// Fungsi Rekursif Faktorial sesuai Modul 3 (ditambah tipe data int agar valid)
int factorial(int number) {
    if (number <= 0) {
        return 1;
    } else {
        return (number * factorial(number - 1));
    }
}

void main() {
    print('=====');
    print('          3.1.5. FUNGSI          ');
    print('=====');
    // Kita uji fungsi rekursif faktorial dengan angka 5
    // Rumus 5! = 5 x 4 x 3 x 2 x 1 = 120
    int inputAngka = 5;
    int hasil = factorial(inputAngka);
    print('Menghitung Faktorial dari angka: $inputAngka');
    print('Hasil dari $inputAngka! adalah : $hasil');
    print('=====');
}
```

Screenshot Output:



```
PS D:\Kuliah Semester 6\Aplikasi Berbasis File Edit View
=====
          3.1.5. FUNGSI
=====
Menghitung Faktorial dari angka: 5
Hasil dari 5! adalah : 120
=====
```

**~SELESAI~**